PhishAware — Task Division & Responsibility Document

Team: Talal · Thameer · Emad

Difficulty Assessment (Based on Actual Line Count)

Backend key files:

- `training`: 2,156 lines (models + views) — heaviest
- `simulations`: 1,813 lines — very heavy
- `analytics`: 1,345 lines (views only!) — heavy
- `campaigns`: 948 lines — medium
- `gamification`: 845 lines — medium
- `community`: 724 lines — medium
- `assessments`: ~800 lines — medium
- `accounts`: 772 lines but conceptually hardest (JWT, RBAC, invitations) — hard
- `notifications`: 459 lines — medium
- `core/companies`: ~300 lines — light

Balanced per-person score: training + campaigns + assessments ≈ **3,904** | simulations + gamification + community + core ≈ **3,882** | accounts + analytics + notifications + companies + setup ≈ **3,000**

Complete File Coverage

Backend — 104 non-trivial files (excluding `__init__.py`, migrations, `venv/`):

Owner	Apps / Files
Talal	Root config (`.env`, `.gitignore`, `manage.py`, `requirements.txt`, `db.sqlite3`, `ARCHITECTURE.md`, `README.md`, `TESTING_GUIDE.md`, `test_signals.py`, `setup_test_data.py`) · `phishaware_backend/` (4 files) · `accounts/` (8 files) · `companies/` (7 files) · `analytics/` (7 files) · `notifications/` (10 files)
Thameer	`simulations/` (11 files) · `core/` (12 files incl. templates + management command) · `gamification/` (9 files) · `community/` (7 files)
Emad	`campaigns/` (7 files) · `assessments/` (7 files) · `training/` (10 files) · `CAMPAIGNS.md`
All three	`ml_models/` (8 files)

> Note: All `__init__.py` files are empty Python package markers auto-generated by Django — owned by whoever owns the app they belong to, no meaningful code inside.

Frontend — 95 files:

Owner	Files

Owner	Files
Talal	All root config + `.env` + `README.md` + `src/assets/react.svg` + `src/api/` + `src/contexts/` + `src/hooks/` + `src/utils/` + `src/routes/` + `src/i18n/` + `src/main.jsx` + `src/App.jsx` + `src/index.css` + `public/` + `pages/auth/` (7 pages)
Thameer	`src/layouts/` · `src/components/common/` + Logo + NotificationDropdown · `pages/admin/` (5 pages) · `pages/company/` (13 pages) · `pages/test/`
Emad	`pages/employee/` (8 pages) · `pages/public/` (9 pages) · `components/training/` (12 interactive components + wrapper)

Backend

Talal

Week 1 — Django Setup & Project Structure

Tasks: Setup SQLite for development · Setup Django · Initialize Django project structure · Create Django apps structure

Files:

- `Backend/manage.py`
- `Backend/requirements.txt`
- `Backend/db.sqlite3`
- `Backend/.env` / `Backend/.env.example`
- `Backend/.gitignore`
- `Backend/phishaware_backend/settings.py`
- `Backend/phishaware_backend/urls.py`
- `Backend/phishaware_backend/wsgi.py`
- `Backend/phishaware_backend/asgi.py`
- `Backend/ARCHITECTURE.md`
- `Backend/README.md`

Week 2 — Authentication, Users & Company Management

Tasks: Create User / Role / Company tables · Implement JWT authentication · Implement RBAC system · Build user management APIs

Files:

- `Backend/apps/accounts/models.py` — `User` (custom AbstractBaseUser), 4 roles, email verification fields (`is_verified`, `verification_token`), invitation fields (`invitation_token`, `invitation_status`)
- `Backend/apps/accounts/views.py` — Login, Register, VerifyEmail, ResendVerification, ForgotPassword, ResetPassword, InviteEmployee, AcceptInvitation, GetInvitationDetails

- `Backend/apps/accounts/serializers.py` — `CustomTokenObtainPairSerializer` (adds role/company_id to JWT payload)
- `Backend/apps/accounts/urls.py`
- `Backend/apps/accounts/urls_employees.py`
- `Backend/apps/accounts/admin.py`
- `Backend/apps/accounts/apps.py`
- `Backend/apps/accounts/tests.py`
- `Backend/apps/companies/models.py` — `Company` model
- `Backend/apps/companies/views.py`
- `Backend/apps/companies/serializers.py`
- `Backend/apps/companies/urls.py`
- `Backend/apps/companies/admin.py`
- `Backend/apps/companies/apps.py`
- `Backend/apps/companies/tests.py`

Extra — Analytics & Notifications

Tasks (not in original sprint list — discovered in codebase): Platform-wide analytics aggregation · Notification service and reminder system

Files:

- `Backend/apps/analytics/models.py`
- `Backend/apps/analytics/views.py` — 1,345 lines of aggregation queries
- `Backend/apps/analytics/serializers.py`
- `Backend/apps/analytics/urls.py`
- `Backend/apps/analytics/admin.py`
- `Backend/apps/analytics/apps.py`
- `Backend/apps/analytics/tests.py`
- `Backend/apps/notifications/models.py`
- `Backend/apps/notifications/views.py`
- `Backend/apps/notifications/serializers.py`
- `Backend/apps/notifications/services.py`
- `Backend/apps/notifications/urls.py`
- `Backend/apps/notifications/admin.py`
- `Backend/apps/notifications/apps.py`
- `Backend/apps/notifications/management/commands/audit_notifications.py`
- `Backend/apps/notifications/management/commands/send_training_reminders.py`
- `Backend/apps/notifications/management/commands/test_notifications.py`
- `Backend/TESTING_GUIDE.md`
- `Backend/test_signals.py`
- `Backend/setup_test_data.py`

> **Difficulty balance:** `accounts` is the most conceptually complex app in the entire project (JWT, RBAC, email verification, invitation system). `analytics/views.py` alone is 1,345 lines. Together these two apps make Talal's workload comparable to the others despite fewer total apps.

Key Techniques & Concepts (Talal — Backend)

Concept	What it is
Django Project Structure	`settings.py` centralizes all configuration (databases, installed apps, email, JWT, CORS). `phishaware_backend/urls.py` is the root URL dispatcher that `include()`s each app's urls.
SQLite for Development	Zero-config file-based database (`db.sqlite3`). Django auto-creates it on first migration. Swap to PostgreSQL in production by changing the `DATABASES` dict in `settings.py`.
Custom User Model	`User` extends `AbstractBaseUser` + `PermissionsMixin`. Email is the login field, not username. `UserManager` provides `create_user()` and `create_superuser()`. Must set `AUTH_USER_MODEL` before first migration.
JWT Authentication	`SimpleJWT` generates access (~5 min) and refresh (~1 day) tokens. `CustomTokenObtainPairSerializer` adds `role`, `company_id`, `user_id` to the token payload — frontend reads role without an extra API call.
RBAC (Role-Based Access Control)	`core/permissions.py` defines `BasePermission` subclasses checking `request.user.role`. 4 roles: `SUPER_ADMIN`, `COMPANY_ADMIN`, `EMPLOYEE`, `PUBLIC_USER`. Views declare `permission_classes = [IsAuthenticated, IsCompanyAdmin]`.
Email Verification Flow	On register → UUID `verification_token` saved → email sent → `/verify-email/<uuid>/` sets `is_verified=True`. Unverified login returns `{"email_not_verified": true}`.
Employee Invitation Flow	`InviteEmployeeView` creates inactive User (`invitation_status='PENDING'`) → email sent → `AcceptInvitationView` activates account, sets `is_verified=True`. Expires after 7 days.
Analytics ORM Aggregation	`analytics/views.py` uses `Count`, `Avg`, `annotate` to compute platform-wide stats. No raw SQL needed — Django ORM translates Python to optimised SQL.
Notification Management Commands	`send_training_reminders` is a Django management command designed to run on a cron schedule to notify employees with overdue training.

Thameer

Week 4 — Email Simulations, Tracking & Core Infrastructure

Tasks: Build email template library CRUD · Implement tracking pixel logic · Implement lure link tracking · Create "You've been caught" landing page · Create RiskScore / RemediationTraining tables · Implement risk scoring algorithm

Files:

- `Backend/apps/simulations/models.py` — `SimulationTemplate` (attack vectors, difficulty), `EmailSimulation`, `TrackingEvent` (OPENED/CLICKED/SUBMITTED), `SimulationCampaign`

(denormalized stats: total_sent, total_clicked, etc.)
- `Backend/apps/simulations/views.py` — 1,166 lines: template CRUD, campaign management, tracking pixel view, lure link redirect view, feedback view
- `Backend/apps/simulations/serializers.py`
- `Backend/apps/simulations/urls.py`
- `Backend/apps/simulations/admin.py`
- `Backend/apps/simulations/apps.py`
- `Backend/apps/simulations/services.py` — `_send_simulation_email()`, `_update_simulation_stats()`, `_update_campaign_stats()`
- `Backend/apps/simulations/tests.py`
- `Backend/apps/simulations/management/commands/seed_simulation_templates.py` — seeds 15 templates (8 EN + 7 AR)
- `Backend/apps/simulations/templates/simulations/landing_page.html`
- `Backend/apps/simulations/templates/simulations/error.html`
- `Backend/apps/core/emails.py` — `send_verification_email`, `send_employee_invitation`, `send_simulation_email`, `send_password_reset_email`
- `Backend/apps/core/permissions.py`
- `Backend/apps/core/models.py`
- `Backend/apps/core/admin.py`
- `Backend/apps/core/apps.py`
- `Backend/apps/core/views.py`
- `Backend/apps/core/tests.py`
- `Backend/apps/core/management/commands/test_email.py`
- `Backend/apps/core/templates/emails/verification.html`
- `Backend/apps/core/templates/emails/invitation.html`
- `Backend/apps/core/templates/emails/password_reset.html`
- `Backend/apps/core/templates/emails/company_welcome.html`

Extra — Gamification & Community

Tasks (from Week 5 + discovered in codebase): Create Leaderboard / Badge / Achievement tables · Implement badge award system · Build leaderboard APIs · Create public content APIs

Files:

- `Backend/apps/gamification/models.py` — `Badge` (8 badge types), `EmployeeBadge`, `PointsTransaction`, `EmployeePoints`
- `Backend/apps/gamification/views.py`
- `Backend/apps/gamification/serializers.py`
- `Backend/apps/gamification/urls.py`
- `Backend/apps/gamification/admin.py`
- `Backend/apps/gamification/apps.py`
- `Backend/apps/gamification/services.py` — 430 lines of badge award logic
- `Backend/apps/gamification/signals.py`
- `Backend/apps/gamification/tests.py`
- `Backend/apps/community/models.py`
- `Backend/apps/community/views.py` — 724 lines of public-facing content APIs

- `Backend/apps/community/serializers.py`
- `Backend/apps/community/urls.py`
- `Backend/apps/community/admin.py`
- `Backend/apps/community/apps.py`
- `Backend/apps/community/tests.py`

> **Difficulty balance:** `simulations/views.py` is 1,166 lines and is architecturally the most complex view file (tracking pixel, email sending, stat updates, redirect logic all in one app). Combined with gamification services logic (~430 lines), this workload equals the others.

Key Techniques & Concepts (Thameer — Backend)

Concept	What it is
Tracking Pixel	A 1×1 transparent PNG served by a Django view. When an email client loads the image it fires an HTTP GET to the server, which logs `TrackingEvent(event_type='OPENED')`.
Lure Link Tracking	Phishing emails contain a Django URL. The view logs `TrackingEvent(event_type='CLICKED')` then redirects to the React `SimulationCaught` page at `/simulation/caught/<token>`.
Denormalized Stats	`SimulationCampaign` stores `total_sent`, `total_clicked`, etc. directly on the model for fast reads. Updated inside `TrackingEvent.save()` via `_update_simulation_stats()` and `_update_campaign_stats()`.
SendGrid SMTP Email	Emails sent via `smtp.sendgrid.net:587`. Django's `EmailMultiAlternatives` sends both plain-text and HTML. `core/emails.py` wraps all four send types into named helper functions.
Django HTML Templates (Email)	`core/templates/emails/` holds HTML email templates rendered via `render_to_string()`. Variables like `{{ verification_url }}` are filled in at send time.
Badge Award Logic	`gamification/services.py` checks badge conditions (e.g. completed 5 trainings → `TRAINING_CHAMPION`). `gamification/signals.py` triggers the service on `post_save` of relevant models.
Public APIs (AllowAny)	`community/` views use `permission_classes = [AllowAny]` — no token needed. Compare this to company views that require `IsAuthenticated` + role checks.
Seed Commands	`seed_simulation_templates` creates 15 phishing templates. Uses `get_or_create()` — safe to run multiple times without creating duplicates (idempotent).

Emad

Week 3 — Campaigns, Training & Quizzes

Tasks: Create Campaign / Quiz / QuizResult tables · Create EmailTemplate / SimulationLog / TrackingEvent tables · Build campaign creation & management APIs · Implement quiz assignment logic · Build quiz taking & submission APIs

Files:

- `Backend/apps/campaigns/models.py` — `Campaign` (DRAFT/ACTIVE/SCHEDULED/COMPLETED/ARCHIVED), `CampaignEmail`, `CampaignAssignment`, `CampaignResult`
- `Backend/apps/campaigns/views.py`
- `Backend/apps/campaigns/serializers.py`
- `Backend/apps/campaigns/urls.py`
- `Backend/apps/campaigns/admin.py`
- `Backend/apps/campaigns/apps.py`
- `Backend/apps/campaigns/tests.py`
- `Backend/apps/assessments/models.py` — `EmailTemplate` (PHISHING/LEGITIMATE, 10 categories), `Quiz`, `QuizQuestion`, `QuizResult`
- `Backend/apps/assessments/views.py`
- `Backend/apps/assessments/serializers.py`
- `Backend/apps/assessments/urls.py`
- `Backend/apps/assessments/admin.py`
- `Backend/apps/assessments/apps.py`
- `Backend/apps/assessments/tests.py`
- `Backend/apps/training/models.py` — `RiskScore` (0–100, 4 levels), `RiskScoreHistory`, `TrainingModule` (3 topics), `TrainingQuestion`, `RemediationTraining`
- `Backend/apps/training/views.py` — 1,240 lines: training assignment, risk score endpoints, remediation logic
- `Backend/apps/training/serializers.py`
- `Backend/apps/training/urls.py`
- `Backend/apps/training/admin.py`
- `Backend/apps/training/apps.py`
- `Backend/apps/training/signals.py` — `post_save` on `TrackingEvent` → `recalculate_score()`
- `Backend/apps/training/tests.py`
- `Backend/apps/training/management/commands/seed_training.py` — 3 modules, 5 bilingual questions each
- `Backend/apps/training/management/commands/clean_training.py`
- `Backend/CAMPAIGNS.md`

> **Difficulty balance:** `training` is the single largest app — `models.py` (916 lines) + `views.py` (1,240 lines) = **2,156 lines**. It contains the core risk scoring algorithm and Django signals wiring. Emad owns exactly Week 3's original sprint scope, which is the most code-dense sprint.

Key Techniques & Concepts (Emad — Backend)

Concept	What it is
Campaign Flow	Company Admin creates a `Campaign` → assigns `EmailTemplate` records (AI-generated phishing + legitimate) → assigns to employees → employees take the quiz (identify which emails are phishing).

Concept	What it is
Quiz Assignment Logic	`CampaignAssignment` links a `Campaign` to a `User`. `QuizResult` stores per-employee scores and answers. The view checks if a quiz is already submitted before allowing re-submission.
Risk Score Algorithm	`RiskScore` (0–100) per employee. Goes UP when employee clicks a phishing link. Goes DOWN when they complete training. Levels: LOW (0–30) / MEDIUM (31–60) / HIGH (61–80) / CRITICAL (81–100).
Django Signals	`training/signals.py` registers a `post_save` receiver on `TrackingEvent`. Every new event automatically calls `recalculate_score()`. Decouples risk scoring from simulation logic. Execution order: `TrackingEvent.save()` → `super().save()` → **signal fires** → then stat update methods.
Auto-Remediation	When `RiskScore` crosses a threshold, `RemediationTraining` is auto-created assigning the employee to a relevant `TrainingModule`. No manual action needed from the company admin.
Seed Commands	`seed_training` creates 3 `TrainingModule` records (EMAIL_SECURITY, MOBILE_SECURITY, SOCIAL_ENGINEERING) each with 5 bilingual questions. Uses `get_or_create()` — idempotent.
DRF Serializers	`ModelSerializer` converts model instances ↔ JSON and validates incoming data. `validate_*()` methods add field-level rules. `create()` / `update()` override default save behaviour.
Training Module Topics	3 seed modules assigned automatically by the remediation engine based on risk level. Each has questions in both English and Arabic.

Frontend

Talal

Week 1 — Project Setup, API Service Layer & Authentication Pages

Tasks: Setup React.js environment · Create axios API service layer · Build authentication pages

Files:

- `Frontend/index.html`
- `Frontend/vite.config.js`
- `Frontend/package.json` / `Frontend/package-lock.json`
- `Frontend/tailwind.config.js` / `Frontend/postcss.config.js` / `Frontend/eslint.config.js`
- `Frontend/.env` / `Frontend/.env.example` / `Frontend/.gitignore` / `Frontend/README.md`
- `Frontend/public/vite.svg`

- `Frontend/public/logo/logo-horizontal.png` (+ logo-vertical, logo-icon, logo-white, logo-email-banner)
- `Frontend/src/main.jsx`
- `Frontend/src/index.css`
- `Frontend/src/App.jsx`
- `Frontend/src/assets/react.svg`
- `Frontend/src/api/axios.js` — base Axios instance, JWT `Authorization` header interceptor, 401 auto-refresh/logout
- `Frontend/src/api/endpoints.js` — all API functions grouped (authAPI, simulationsAPI, campaignsAPI, employeesAPI, etc.)
- `Frontend/src/api/index.js`
- `Frontend/src/api/notifications.js`
- `Frontend/src/contexts/AuthContext.jsx` — global user/token/role state, login/logout functions
- `Frontend/src/contexts/ThemeContext.jsx` — dark/light mode, persists to localStorage
- `Frontend/src/contexts/index.js`
- `Frontend/src/hooks/useApi.js` — wraps API calls with `loading`, `error`, `data` state
- `Frontend/src/hooks/index.js`
- `Frontend/src/utils/helpers.js`
- `Frontend/src/utils/index.js`
- `Frontend/src/routes/ProtectedRoute.jsx` — checks auth + role before rendering
- `Frontend/src/routes/index.jsx` — full route tree for all roles
- `Frontend/src/i18n/ar.json`
- `Frontend/src/i18n/en.json`
- `Frontend/src/i18n/index.js`
- `Frontend/src/pages/auth/LoginPage.jsx`
- `Frontend/src/pages/auth/RegisterPage.jsx`
- `Frontend/src/pages/auth/RegisterCompany.jsx`
- `Frontend/src/pages/auth/ForgotPassword.jsx`
- `Frontend/src/pages/auth/ResetPassword.jsx`
- `Frontend/src/pages/auth/VerifyEmailPage.jsx`
- `Frontend/src/pages/auth/AcceptInvitation.jsx`

Key Techniques & Concepts (Talal — Frontend)

Concept	What it is
Vite	Ultra-fast build tool. `vite.config.js` sets up dev-server proxy (`/api` → Django), path aliases (`@` = `src/`), build output to `dist/`. `npm run dev` starts dev server; `npm run build` bundles for production.
Tailwind CSS	Utility-first CSS. No separate `.css` files per component — styles are class names in JSX: `className="bg-blue-600 text-white px-4 py-2 rounded-lg dark:bg-blue-500"`. Dark mode via `dark:` prefix.
Axios Interceptors	`axios.js` attaches `Authorization: Bearer <token>` to every request automatically. If a 401 response arrives, the interceptor attempts a silent token refresh, then logs out if refresh fails.

Concept	What it is
React Context API	`AuthContext` is a global store for the logged-in user. Wrapped around the entire app in `App.jsx`. Any component calls `useContext(AuthContext)` to read `user`, `role`, `token` without prop drilling.
Protected Routes	`ProtectedRoute.jsx` reads `AuthContext`. No user → redirect to `/login`. Wrong role → redirect to `/unauthorized`. Wraps all authenticated route groups in `index.jsx`.
React Router v6	Declarative client-side routing. `<Outlet />` in layout components renders child routes. `useNavigate()` redirects programmatically. `useParams()` reads `:token` from URL. No page reloads.
i18n Bilingual Support	`ar.json` and `en.json` hold key-value string pairs. `i18n/index.js` exports `t(key, lang)`. Arabic pages apply `dir="rtl"` for right-to-left layout. Language preference read from user profile.
Custom Hook `useApi`	Wraps API calls with `loading`, `error`, `data` state so each page avoids duplicating try/catch + useState boilerplate. Returns `{ data, loading, error, execute }`.

Thameer

Week 2 — Layouts, Components, Admin & Company Management UI

Tasks: Build admin main layout · Create campaign management UI · Build template library UI · Create admin analytics dashboard · Build employee management UI · Implement AI email generation UI

Files:

- `Frontend/src/layouts/DashboardLayout.jsx` — sidebar + topbar shell, uses `<Outlet />` for child pages
- `Frontend/src/layouts/PublicLayout.jsx` — navbar + footer for public pages
- `Frontend/src/layouts/index.js`
- `Frontend/src/components/Logo.jsx`
- `Frontend/src/components/NotificationDropdown.jsx`
- `Frontend/src/components/index.js`
- `Frontend/src/components/common/Button.jsx`
- `Frontend/src/components/common/Input.jsx`
- `Frontend/src/components/common/LoadingSpinner.jsx`
- `Frontend/src/components/common/ThemeToggle.jsx`
- `Frontend/src/components/common/index.js`
- `Frontend/src/pages/admin/AdminDashboard.jsx`
- `Frontend/src/pages/admin/CompanyCreate.jsx`
- `Frontend/src/pages/admin/CompanyList.jsx`
- `Frontend/src/pages/admin/PlatformAnalytics.jsx` — 1,104 lines of charts and aggregated stats
- `Frontend/src/pages/admin/UserManagement.jsx`
- `Frontend/src/pages/company/CompanyDashboard.jsx`
- `Frontend/src/pages/company/CompanyProfile.jsx`

- `Frontend/src/pages/company/CompanyAnalytics.jsx`
- `Frontend/src/pages/company/CompanyEmployees.jsx` — 1,303 lines: invite modal, employee table, role management
- `Frontend/src/pages/company/EmployeeList.jsx`
- `Frontend/src/pages/company/CampaignCreate.jsx`
- `Frontend/src/pages/company/CampaignList.jsx`
- `Frontend/src/pages/company/CampaignDetails.jsx`
- `Frontend/src/pages/company/SimulationCreate.jsx`
- `Frontend/src/pages/company/SimulationList.jsx`
- `Frontend/src/pages/company/CompanySimulations.jsx`
- `Frontend/src/pages/company/SimulationAnalytics.jsx` — live auto-refresh every 15 seconds
- `Frontend/src/pages/company/TrainingManagement.jsx`
- `Frontend/src/pages/test/NotificationTest.jsx`

Key Techniques & Concepts (Thameer — Frontend)

Concept	What it is
Layout Pattern (`<Outlet />`)	`DashboardLayout.jsx` renders sidebar and topbar once. The current page is injected via React Router's `<Outlet />`. All `/company/*` and `/admin/*` routes get the shell without repeating layout code.
Reusable Component Library	`components/common/` (Button, Input, LoadingSpinner, ThemeToggle) used across all pages. Changing one component updates everywhere. Props like `variant`, `size`, `disabled` control appearance.
Dark Mode Toggle	`ThemeToggle.jsx` calls `ThemeContext.toggle()` which adds/removes the `dark` class on `<html>`. Tailwind's `dark:bg-gray-900` activates automatically. Preference saved to `localStorage`.
Simulation Analytics Auto-Refresh	`SimulationAnalytics.jsx` uses `useEffect` with `setInterval(fetchData, 15000)` to poll the API every 15 seconds. The cleanup function `clearInterval()` prevents memory leaks on unmount.
Invite Employee Flow	`CompanyEmployees.jsx` opens a modal form → calls `employeesAPI.invite({name, email, department})` → backend creates inactive user + sends invitation email → table refreshes automatically.
AI Email Generation UI	Company admin page sends parameters to the backend ML endpoint and displays the AI-generated phishing email in a preview panel. The template can then be saved to the library.
Campaign Management Workflow	Multi-step: select AI-generated + legitimate templates → assign to employees → activate → employees take quiz → view results in `CampaignDetails.jsx`.
Notification Dropdown	`NotificationDropdown.jsx` polls the notifications API and shows an unread count badge. Clicking marks as read and navigates to the relevant resource.

Emad

Week 3 — Employee Dashboard, Gamification UI, Public Portal & Interactive Training

Tasks: Build employee dashboard · Create quiz interface · Implement gamification display · Build public community portal · Create awareness content pages · Build public quiz interface

Files:

- `Frontend/src/pages/employee/EmployeeDashboard.jsx`
- `Frontend/src/pages/employee/EmployeeProfile.jsx`
- `Frontend/src/pages/employee/EmployeeTraining.jsx`
- `Frontend/src/pages/employee/TakeTraining.jsx`
- `Frontend/src/pages/employee/EmployeeQuizzes.jsx`
- `Frontend/src/pages/employee/TakeQuiz.jsx` — 1,768 lines: per-question state machine, answer tracking, score submission
- `Frontend/src/pages/employee/EmployeeLeaderboard.jsx`
- `Frontend/src/pages/employee/EmployeeBadges.jsx`
- `Frontend/src/pages/public/LandingPage.jsx`
- `Frontend/src/pages/public/PublicHome.jsx`
- `Frontend/src/pages/public/CommunityPortal.jsx`
- `Frontend/src/pages/public/TrainingTopics.jsx`
- `Frontend/src/pages/public/TopicTraining.jsx`
- `Frontend/src/pages/public/PublicQuiz.jsx`
- `Frontend/src/pages/public/SimulationCaught.jsx` — "You've been caught" page, fetches red_flags + explanation via token
- `Frontend/src/pages/public/NotFoundPage.jsx`
- `Frontend/src/pages/public/UnauthorizedPage.jsx`
- `Frontend/src/components/training/InteractiveLessonWrapper.jsx`
- `Frontend/src/components/training/interactive/public/PhishAware_V1_EN.jsx`
- `Frontend/src/components/training/interactive/public/PhishAware_V1_AR.jsx`
- `Frontend/src/components/training/interactive/public/PhishAware_V2_EN.jsx`
- `Frontend/src/components/training/interactive/public/PhishAware_V2_AR.jsx`
- `Frontend/src/components/training/interactive/public/PhishAware_V3_EN.jsx`
- `Frontend/src/components/training/interactive/public/PhishAware_V3_AR.jsx`
- `Frontend/src/components/training/interactive/employee/PhishAware_V4_EN.jsx`
- `Frontend/src/components/training/interactive/employee/PhishAware_V4_AR.jsx`
- `Frontend/src/components/training/interactive/employee/PhishAware_V5_EN.jsx`
- `Frontend/src/components/training/interactive/employee/PhishAware_V5_AR.jsx`
- `Frontend/src/components/training/interactive/employee/PhishAware_V6_EN.jsx`
- `Frontend/src/components/training/interactive/employee/PhishAware_V6_AR.jsx`

> **Difficulty balance:** `TakeQuiz.jsx` is 1,768 lines — the largest single frontend file. The 12 interactive training components add ~3,600 more lines. Emad's frontend total is the highest (~5,000 lines) but the training components are content-heavy rather than architecturally complex.

Key Techniques & Concepts (Emad — Frontend)

Concept	What it is
SimulationCaught Page	Public route at `/simulation/caught/:token` (no login required). When an employee clicks a phishing link they land here. The token calls `GET /api/v1/simulations/feedback/{token}/` which returns `red_flags` and `explanation` — an immediate teachable moment.
Interactive Lesson Components	Each `PhishAware_VX_XX.jsx` is a self-contained multi-step lesson (animated slides, scenarios, embedded mini-quizzes). V1–V3 are public (no auth). V4–V6 are for authenticated employees. `InteractiveLessonWrapper.jsx` handles API progress-saving around any version.
TakeQuiz State Machine	`TakeQuiz.jsx` renders one question at a time using an index state. Answers stored in a local array. On the last question the full answer array is POSTed. Score and feedback displayed after submission.
Gamification Display	`EmployeeLeaderboard.jsx` shows a ranked table with points. `EmployeeBadges.jsx` shows earned badges (highlighted) vs locked badges (greyed out) in a grid. Both fetch from the gamification API.
Public Community Portal	`CommunityPortal.jsx` and `TrainingTopics.jsx` use no-auth API calls — no JWT token attached. They provide phishing awareness content to the general public without registration.
RTL Arabic Support	Arabic lesson components (`_AR.jsx`) set `dir="rtl"` on their root element. Text aligns right. Arabic strings come from `i18n/ar.json`. Both EN and AR versions loaded by `InteractiveLessonWrapper` based on the user's language preference.
Risk Score Visual	`EmployeeDashboard.jsx` displays the employee's `RiskScore` (0–100) as a colour-coded indicator: green (LOW 0–30), yellow (MEDIUM 31–60), orange (HIGH 61–80), red (CRITICAL 81–100).
useEffect Cleanup Pattern	Any `useEffect` that sets up timers returns a cleanup function: `return () => clearInterval(id)`. Without this, intervals keep running after the user navigates away, causing memory leaks and ghost API calls.

Django — Features & Functions Used in This Project

What is Django?

Django is a Python web framework that follows the **MVT pattern** (Model–View–Template). In this project it's used as a **pure API backend** — no HTML pages rendered by Django except email templates and one simulation landing page. The frontend is a completely separate React app.

---

1. Django ORM (Object Relational Mapper)

The ORM lets you write Python classes and Django automatically creates and manages the database tables. You never write raw SQL.

```
# Defining a table
class RiskScore(models.Model):
    employee = models.OneToOneField(User, on_delete=models.CASCADE)
    score = models.IntegerField(default=50)

# Querying
RiskScore.objects.filter(score__gte=61)        # WHERE score >= 61
RiskScore.objects.get(employee=user)            # exact match
RiskScore.objects.select_related('employee')    # JOIN in one query
```

Used for: every model in all 11 apps. Key field types: `CharField`, `IntegerField`, `BooleanField`, `UUIDField`, `DateTimeField`, `ForeignKey`, `OneToOneField`, `ManyToManyField`.

---

2. Custom User Model (AbstractBaseUser)

Django's built-in User uses a username. We replaced it entirely with an email-based User.

```
class User(AbstractBaseUser, PermissionsMixin):
    email = models.EmailField(unique=True)
    role = models.CharField(choices=ROLE_CHOICES)
    USERNAME_FIELD = 'email'   # login with email, not username
```

Key rule: `AUTH_USER_MODEL = 'accounts.User'` must be set in `settings.py` before the first migration. Changing it later breaks everything.

---

3. Django REST Framework (DRF)

DRF is a library that turns Django views into a full REST API with JSON responses.

Serializers — convert model instances ↔ JSON, and validate incoming data:

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'email', 'role']
```

ViewSets — one class handles GET (list), POST (create), PUT/PATCH (update), DELETE:

```
class SimulationCampaignViewSet(viewsets.ModelViewSet):
    queryset = SimulationCampaign.objects.all()
    serializer_class = SimulationCampaignSerializer
    permission_classes = [IsAuthenticated, IsCompanyAdmin]
```

APIView — for custom logic that doesn't fit standard CRUD:

```
class AcceptInvitationView(APIView):
    def post(self, request, token):
        user = get_object_or_404(User, invitation_token=token)
        user.is_active = True
        user.save()
```

---

4. JWT Authentication (SimpleJWT)

JSON Web Tokens are stateless authentication tokens. No session or cookie needed.

- **Access token** — short-lived (~5 min). Sent in every request: `Authorization: Bearer <token>`
- **Refresh token** — long-lived (~1 day). Used only to get a new access token

We customized the token to carry extra claims:

```
class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        token['role'] = user.role
        token['company_id'] = str(user.company_id)
        return token
```

The React frontend reads the role from the token itself — no extra API call needed.

---

5. RBAC — Role-Based Access Control

Custom `BasePermission` classes in `Backend/apps/core/permissions.py` check `request.user.role` on every request.

```
class IsCompanyAdmin(BasePermission):
    def has_permission(self, request, view):
        return request.user.role in ['COMPANY_ADMIN', 'SUPER_ADMIN']
```

Views declare which roles are allowed:

```
permission_classes = [IsAuthenticated, IsCompanyAdmin]
```

`AllowAny` is used on public community endpoints (no login required).

---

6. Django Signals

Signals let one part of the code react to events in another part **without direct coupling**.

```
# training/signals.py
@receiver(post_save, sender=TrackingEvent)
def update_risk_on_tracking_event(sender, instance, created, **kwargs):
    if created:
        instance.employee.riskScore.recalculate_score()
```

Used for: risk score recalculation whenever a `TrackingEvent` is saved, and badge award checks in `gamification/signals.py`.

Execution order in this project: `TrackingEvent.save()` → `super().save()` → **post_save signal fires** → then `_update_simulation_stats()` → then `_update_campaign_stats()`

---

7. Migrations

Migrations are version-controlled snapshots of your database schema. Every time you change a model, you run:

```
python manage.py makemigrations   # creates migration file
python manage.py migrate          # applies it to the database
```

Key migrations in this project:

- `accounts/0002_email_verification.py` — adds verification fields + backfills `is_verified=True` for existing users
- `accounts/0003_invitation_fields.py` — adds invitation fields

---

8. Management Commands

Custom CLI commands that extend `python manage.py`. Each inherits from `BaseCommand` and implements `handle()`.

Command	What it does
`seed_simulation_templates`	Creates 15 phishing templates (8 EN + 7 AR)
`seed_training`	Creates 3 modules with 5 bilingual questions each
`test_email`	Sends a test email via SendGrid to verify config
`send_training_reminders`	Notifies employees with overdue training
`audit_notifications`	Checks and cleans up stale notifications

Command	What it does
`clean_training`	Removes seed training data (reverse of seed_training)
`test_notifications`	Developer tool to generate test notification records

---

9. Django Admin

Auto-generated web UI at `/admin/` for managing all models. Each app registers its models:

```
# simulations/admin.py
@admin.register(SimulationTemplate)
class SimulationTemplateAdmin(admin.ModelAdmin):
    list_display = ['name', 'attack_vector', 'difficulty']
    list_filter = ['difficulty', 'language']
```

Used during development to inspect and manage data without writing custom views.

---

10. Email — EmailMultiAlternatives + SendGrid

Django's email backend sends through SendGrid SMTP (`smtp.sendgrid.net:587`). `core/emails.py` wraps all sends:

```
msg = EmailMultiAlternatives(subject, text_body, from_email, [to_email])
msg.attach_alternative(html_body, "text/html")   # attach HTML version
msg.send()
```

HTML templates live in `core/templates/emails/`. Django's `render_to_string()` fills in variables like `{{ verification_url }}`.

---

11. Settings & Environment Variables

`phishaware_backend/settings.py` reads secrets from `.env` via `python-decouple`:

```
SECRET_KEY = config('SECRET_KEY')
EMAIL_HOST_PASSWORD = config('SENDGRID_API_KEY')
FRONTEND_URL = config('FRONTEND_URL')
```

No secrets are committed to git. `.env.example` shows which keys are needed without real values.

---

12. URL Routing Structure

```
phishaware_backend/urls.py       ← root router
    ├── api/v1/auth/             → accounts/urls.py
    ├── api/v1/employees/        → accounts/urls_employees.py
    ├── api/v1/simulations/      → simulations/urls.py
    ├── api/v1/training/         → training/urls.py
    ├── api/v1/campaigns/        → campaigns/urls.py
    ├── api/v1/assessments/      → assessments/urls.py
    ├── api/v1/analytics/        → analytics/urls.py
    ├── api/v1/gamification/     → gamification/urls.py
    ├── api/v1/community/        → community/urls.py
    ├── api/v1/notifications/    → notifications/urls.py
    └── api/v1/companies/        → companies/urls.py
```

DRF `DefaultRouter` auto-generates list/detail URLs for ViewSets.

React / Frontend — Features & Functions Used in This Project

What is React?

React is a JavaScript library for building UIs from reusable components. Each component is a function that returns JSX (HTML-like syntax in JavaScript) and re-renders automatically when its state or props change.

---

1. Vite (Build Tool)

Vite replaces Create React App. It's significantly faster because it serves files using native ES modules during development instead of bundling everything upfront.

- `vite.config.js` — sets up: dev server proxy (`/api` → Django), path aliases (`@` = `src/`), build output to `dist/`
- `npm run dev` — starts dev server with hot module reload
- `npm run build` — bundles everything into `dist/` for deployment

---

2. JSX & Component Structure

Every page and UI element is a React component — a JavaScript function that returns JSX:

```jsx
function LoginPage() {
  const [email, setEmail] = useState('')

  return (
    <div className="flex flex-col gap-4">
      <Input value={email} onChange={e => setEmail(e.target.value)} />
      <Button onClick={handleLogin}>Login</Button>
```

```
      </div>
    )
  }
```

Props pass data down to children. Events (`onClick`, `onChange`) call handler functions.

---

3. React Hooks Used

Hook	Purpose	Where used
`useState`	Local state (form values, loading, error)	Every page
`useEffect`	Side effects: fetch data on mount, set up timers	Every page that fetches data
`useContext`	Read from Context (auth, theme)	All authenticated pages
`useNavigate`	Programmatic routing (redirect after login)	Auth pages
`useParams`	Read URL params (`/verify-email/:token`)	VerifyEmailPage, SimulationCaught, AcceptInvitation
`useCallback`	Memoize functions to avoid unnecessary re-renders	Performance-sensitive pages
`useRef`	Reference DOM elements, persist values without re-render	Forms, timers

---

4. React Router v6

Declarative client-side routing — no page reloads when navigating.

```jsx
// routes/index.jsx
<Routes>
  <Route path="/" element={<PublicLayout />}>
    <Route index element={<LandingPage />} />
    <Route path="login" element={<LoginPage />} />
  </Route>
  <Route path="/company" element={
    <ProtectedRoute role="COMPANY_ADMIN">
      <DashboardLayout />
    </ProtectedRoute>
  }>
    <Route path="dashboard" element={<CompanyDashboard />} />
  </Route>
</Routes>
```

`<Outlet />` inside layout components is where child routes render. `useNavigate()` redirects programmatically. `useParams()` reads `:token` from the URL.

5. Axios & API Layer

`axios.js` creates a configured instance used everywhere:

```
const api = axios.create({ baseURL: import.meta.env.VITE_API_URL })

// Request interceptor — auto-attach token
api.interceptors.request.use(config => {
  const token = localStorage.getItem('access_token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

// Response interceptor — auto-handle 401
api.interceptors.response.use(null, async error => {
  if (error.response?.status === 401) {
    // attempt token refresh, or logout
  }
})
```

`endpoints.js` organizes all API calls by domain:

```
export const simulationsAPI = {
  list: () => api.get('/simulations/'),
  send: (id) => api.post(`/simulations/${id}/send/`),
  getSimulationFeedback: (token) => api.get(`/simulations/feedback/${token}/`),
}
```

6. Context API (Global State)

React Context solves the problem of passing props through many levels of components.

AuthContext — stores the logged-in user globally:

```
const { user, token, role, login, logout } = useContext(AuthContext)
```

ThemeContext — stores dark/light mode preference. When toggled, adds/removes `dark` class on `<html>` and saves to `localStorage`. Tailwind's `dark:` prefix activates automatically.

7. Tailwind CSS

Utility-first CSS — styles are class names applied directly in JSX. No separate `.css` files per component.

```
<button className="bg-blue-600 hover:bg-blue-700 text-white px-4 py-2 rounded-lg
                   dark:bg-blue-500 transition-colors duration-200">
  Send Simulation
</button>
```

Key Tailwind features used:

- `dark:` — dark mode variants
- `hover:`, `focus:` — state variants
- `md:`, `lg:` — responsive breakpoints
- `flex`, `grid` — layout utilities
- Custom values in `tailwind.config.js` (brand colors, fonts)

---

8. Protected Routes & Role Guards

`ProtectedRoute.jsx` wraps any route that requires authentication:

```
function ProtectedRoute({ children, role }) {
  const { user } = useContext(AuthContext)
  if (!user) return <Navigate to="/login" />
  if (role && user.role !== role) return <Navigate to="/unauthorized" />
  return children
}
```

If someone types `/company/dashboard` without being logged in, they are immediately redirected to `/login`.

---

9. i18n — Bilingual Support (English + Arabic)

`ar.json` and `en.json` hold key-value string pairs. `i18n/index.js` exports a `t(key, lang)` lookup function. Arabic pages apply `dir="rtl"` for right-to-left layout. The user's language preference is read from their profile.

---

10. useEffect for Auto-Refresh (Polling)

`SimulationAnalytics.jsx` shows live stats that update every 15 seconds:

```
useEffect(() => {
  fetchStats()                              // load immediately
  const id = setInterval(fetchStats, 15000) // then every 15s
  return () => clearInterval(id)            // cleanup on unmount
}, [simulationId])
```

The cleanup function (`return () => clearInterval(id)`) is critical — without it, the interval keeps running after the user navigates away, causing memory leaks and ghost API calls.

---

11. Interactive Training Components Pattern

Each `PhishAware_VX_XX.jsx` is a self-contained multi-step lesson. The pattern:

- Local `step` state tracks which screen is shown
- Conditional rendering shows different content per step
- V1–V3: public, no auth required
- V4–V6: employee, progress is saved to the API
- `InteractiveLessonWrapper.jsx` wraps any version and handles the API calls for recording completion

---

12. Environment Variables in React

Vite exposes env vars prefixed with `VITE_`:

```
// .env
VITE_API_URL=http://localhost:8000/api/v1

// In code
axios.create({ baseURL: import.meta.env.VITE_API_URL })
```

Unlike backend `.env`, these are **baked into the build** — they become part of the JavaScript bundle. Never put secrets here.

AI / ML — Shared by All Three Students

These files are equally the responsibility of Talal, Thameer, and Emad.

Files:

- `Backend/ml_models/lstm_model.py` — LSTM model class definition, `forward()` method, loading `.pth` weights
- `Backend/ml_models/email_generator.py` — uses the loaded model to classify or generate phishing email text
- `Backend/ml_models/vocabulary.py` — tokenization: converts text → integer sequences the model can process
- `Backend/ml_models/model_config.json` — hyperparameters (vocab size, hidden units, layers, etc.)
- `Backend/ml_models/vocab_en.json` — English word-to-index mapping
- `Backend/ml_models/vocab_ar.json` — Arabic word-to-index mapping
- `Backend/ml_models/phishing_lstm_en.pth` — pre-trained English model weights (PyTorch)
- `Backend/ml_models/phishing_lstm_ar.pth` — pre-trained Arabic model weights (PyTorch)

Key Concepts:

Concept	Explanation
LSTM (Long Short-Term Memory)	A type of Recurrent Neural Network (RNN) designed for sequential data like text. Unlike a simple RNN, an LSTM has a "memory cell" that can hold information over long sequences, making it good at understanding context in sentences.
`.pth` **File (PyTorch weights)**	Contains the trained model's learned parameters (millions of numbers). The model architecture is defined in `lstm_model.py`; the `.pth` file fills in the weights. Loading: `model.load_state_dict(torch.load('phishing_lstm_en.pth'))`.
Vocabulary / Tokenization	Text cannot be fed directly into a neural network. Each word is mapped to an integer ID using `vocab_en.json` (e.g., "urgent" → 342). The model sees a sequence of numbers, not words.
Two Separate Models	One model trained on English phishing emails, one on Arabic. The vocabulary and language patterns are completely different between the two languages — a single bilingual model would underperform both.
Use in the Platform	Called from `email_generator.py` when a company admin requests an AI-generated phishing simulation email. Generates realistic email text matching a chosen attack vector (urgency, authority, prize, etc.).